DVR & Dr. HS MIC COLLEGE OF TECHNOLOGY

(An Autonomous Institution | Approved by AICTE, New Delhi | Accredited by NAAC 'A+' Grade)

Kanchikacherla, NTR District, Andhra Pradesh - 521180

OPERATING SYSTEMS

B.TECH V SEMESTER (AI&ML / CSE) — REGULATION MIC23

Course Code: 23AM5T03 / 23CS4T01 | Academic Year 2024–2026

COMPREHENSIVE MASTER EXAM PREPARATION NOTES

Complete Question-by-Question Faculty Answers with Standard Exam Diagrams,
Intuitive Concept-Flow Charts, Step-by-Step Numerical Solutions, and Comparison Tables.

(Covering 100% of Syllabus, Model Papers, University Exams & Mid Sheets)

PRIMARY REFERENCE SOURCES INTEGRATED:

1. Official Autonomous Syllabus Copy (23AM5T03 w.e.f. 2023-24)
2. Official Lecture Material & Faculty Unit Notes (Units I, II, III, IV, V)
3. University Supplementary Examinations Question Paper (April/May 2026)
4. University Model Examination Papers (Model Paper-I & Model Paper-II)
5. Mid-Term 1 & Mid-Term 2 Important Question Sheets & Question Banks
6. Units 3, 4, 5 Numerical Problems & Case Studies Sheet

PREPARED BY: SENIOR OPERATING SYSTEMS FACULTY & EXAM SPECIALIST

Designed for Clarity, High Examination Retention, and Maximum Scoring Potential

EXAMINATION ARCHITECTURE & UNIT-WISE CONTENTS

This Master Study Material is organized strictly Unit-Wise in accordance with the official MIC23 curriculum. Each unit contains 2-Mark short answers, 5-Mark detailed conceptual answers, 10-Mark comprehensive essay answers, and complete step-by-step numerical calculations. Every question includes a dedicated, clean black-and-white exam diagram.

Syllabus Units & Question Weightage Breakdown:

Unit	Unit Title	2-Mark Qs	5-Mark Qs	10-Mark LAQ	Numerical Problems
Unit I	Operating Systems Overview & Structures	14 Questions	16 Questions	2 In-depth LAQs	N/A (Theory Only)
Unit II	Processes, Threads & CPU Scheduling	13 Questions	11 Questions	2 In-depth LAQs	5 Solved Problems
Unit III	Synchronization Tools & Deadlocks	16 Questions	13 Questions	2 In-depth LAQs	2 Solved Problems
Unit IV	Memory Management & Mass Storage	10 Questions	12 Questions	2 In-depth LAQs	7 Solved Problems
Unit V	File System & Protection Mechanisms	7 Questions	13 Questions	2 In-depth LAQs	Case-based Analysis

High-Frequency Repeated Questions Matrix (Top Exam Targets):
The following topics appear repeatedly across the University Supplementary Exam (2026), Model Papers I & II, and Mid Question Sheets. They represent guaranteed questions for university examinations:

Topic / Question Focus	Appeared In	Marks	Unit
Functions & Services of Operating System	Univ 2026, Model-I, Model-II, Mid-1	5M / 10M	Unit I
OS Structures (Layered, Microkernel, Monolithic)	Univ 2026, Model-I, Mid-1	5M / 10M	Unit I
Process State Transition Diagram & PCB	Model-II, Mid-1, Class Tests	5M / 10M	Unit II
Multithreading Models & Inter-Process Comm (IPC)	Univ 2026, Model-II, Mid-1	5M	Unit II
CPU Scheduling (FCFS, SJF, Round Robin with Gantt)	Univ 2026, Model-I, Mid-1	5M / 10M	Unit II
Dining Philosophers Problem using Semaphores	Univ 2026, Model-I, Model-II, Mid-1	5M / 10M	Unit III
Banker's Algorithm & Deadlock Avoidance	Univ 2026, Mid-2 QB, Problems Sheet	5M / 10M	Unit III
Four Necessary Deadlock Conditions & Prevention	Univ 2026, Model-I, Model-II, Mid-2	2M / 5M	Unit III
Demand Paging & Page Fault Handling Routine	Univ 2026, Model-I, Mid-2 QB	5M / 10M	Unit IV
Page Replacement (FIFO, Optimal, LRU reference strings)	Model-II, Mid-2 QB, Problems Sheet	5M / 10M	Unit IV
Disk Scheduling (FCFS, SSTF, SCAN, LOOK, C-SCAN)	Univ 2026, Model-II, Mid-2 QB	5M / 10M	Unit IV
File Allocation Methods (Contiguous, Linked, Indexed)	Univ 2026, Model-I, Mid-2 QB	5M / 10M	Unit V
Protection Goals, Access Matrix & Protection Rings	Model-I, Model-II, Mid-2 QB	5M / 10M	Unit V

UNIT - I: OPERATING SYSTEMS OVERVIEW & SYSTEM STRUCTURES

SECTION A: 2-MARK QUESTIONS (Short, Direct Answers)

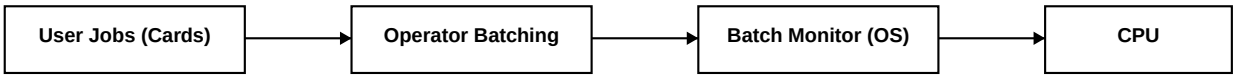
Q1. Define batch processing system. [2M]

(Source: University Exam 2026)

A batch processing system is an early operating system where similar jobs are grouped together into "batches" and executed one after another without any direct user interaction during execution.

- **Working Principle:** Users prepare jobs off-line (on punched cards or tapes) and submit them to a computer operator who sorts and batches them for the CPU.
- **Advantage:** Reduces operator setup time and keeps CPU busy between job transitions.

Fig 1.1: Batch Processing System Concept Flow



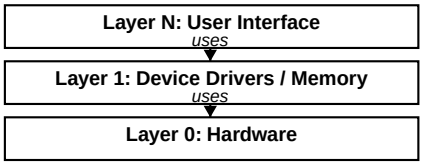
Q2. What are layered operating systems? [2M]

(Source: University Exam 2026)

A layered operating system divides the system into multiple separate levels or layers. Layer 0 is the computer hardware, and the highest layer (Layer N) is the user interface. Each layer is built strictly on top of lower layers and uses services of only the layer immediately below it.

- **Key Benefit:** Modularity and easy debugging; if an error occurs, it must be in the layer being tested.

Fig 1.2: Layered Operating System Architecture



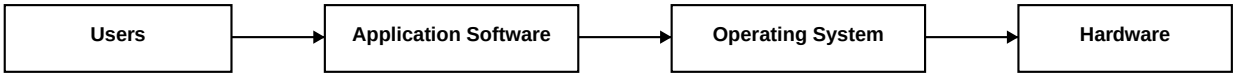
Q3. What is an Operating System? [2M]

(Source: Model Paper-I / Mid-1 Imp)

An Operating System (OS) is a system program that acts as an intermediary or bridge between a computer user and computer hardware. It manages system hardware resources and provides an environment for user programs to execute conveniently and efficiently.

- **Core Goals:** 1. Execute user programs; 2. Make computer convenient to use; 3. Use hardware efficiently.

Fig 1.3: Abstract View of Computer System Components



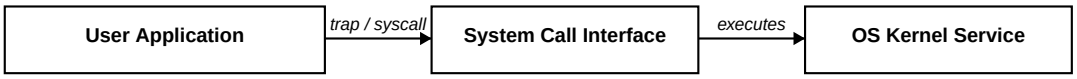
Q4. What are system calls in operating systems? / Define system calls. [2M]

(Source: Model Paper-I / Mid-1 Imp)

A system call is the programmatic interface provided by the operating system that allows a user-level running program to request privileged services directly from the OS kernel (e.g., file read/write, process creation, hardware access).

- **Mechanism:** It triggers a software interrupt (trap) switching the processor from user mode to kernel mode.

Fig 1.4: System Call Invocation Interface



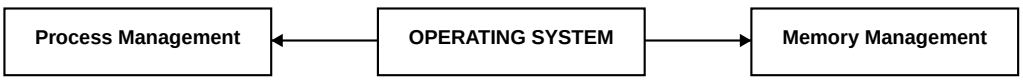
Q5. List any two functions of an operating system. [2M]

(Source: Model Paper-II)

Two fundamental functions of an operating system are:

- **Process Management:** Creating, scheduling, synchronizing, and terminating processes on the CPU.
- **Memory Management:** Tracking parts of physical memory in use, allocating memory blocks, and deallocating memory when programs exit.

Fig 1.5: Core Functions of Operating System



Q6. What is the difference between open-source and proprietary operating systems? [2M]

(Source: Model Paper-II)

An open-source OS provides its source code freely to the public for study, modification, and enhancement (e.g., Linux, FreeBSD). A proprietary (closed-source) OS keeps its source code strictly private and commercialized; users only receive compiled binaries (e.g., Microsoft Windows, macOS).

Feature	Open-Source OS	Proprietary OS
Source Code	Freely accessible and modifiable by all	Hidden, closed, and legally protected
Cost & License	Usually free under GPL/BSD licenses	Commercial license fee required
Examples	GNU/Linux, Ubuntu, Android, FreeBSD	Microsoft Windows, macOS, iOS

Q7. Define Kernel. [2M]

(Source: Mid-1 Imp)

The Kernel is the central core program of an operating system that remains loaded in main memory at all times after booting. It runs in privileged (kernel) mode and has direct, unrestricted control over all computer hardware components.

Fig 1.6: Position of Kernel in Computing Hierarchy

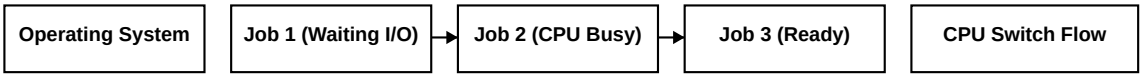


Q8. What is multiprogramming? [2M]

(Source: Mid-1 Imp)

Multiprogramming is a technique where multiple jobs are kept in main memory simultaneously. When the currently executing job needs to wait for an I/O operation to complete, the operating system immediately switches the CPU to execute another job, thereby preventing the CPU from sitting idle.

Fig 1.7: Multiprogramming Memory Layout



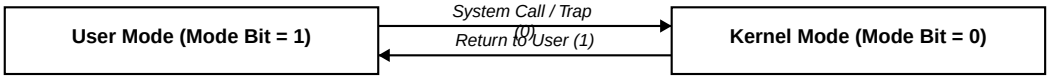
Q9. How Dual-Mode Operation works? [2M]

(Source: Mid-1 Imp)

Dual-mode operation protects the operating system and other programs from erroneous or malicious user programs. Hardware provides a "mode bit":

- **Mode Bit = 0 (Kernel Mode):** The OS executes privileged hardware instructions.
- **Mode Bit = 1 (User Mode):** User programs execute with restricted hardware access.

Fig 1.8: Dual-Mode Operation Switching



Q10. What are the objectives of operating systems? [2M]

(Source: Mid-1 Imp)

The three primary objectives of any operating system are:

- **Convenience:** Makes the computer system user-friendly and easier to operate.
- **Efficiency:** Enables computer resources (CPU, RAM, Disk) to be utilized at maximum throughput.
- **Ability to Evolve:** Allows easy development, testing, and introduction of new features without disrupting existing services.

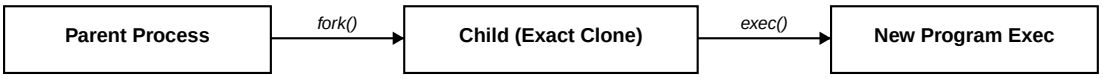
Q11. What is the use of fork and exec system calls? [2M]

(Source: Mid-1 Imp)

`fork()` and `exec()` are foundational UNIX/Linux process creation system calls:

- **fork():** Creates a new duplicate child process that is an exact replica of the parent process.
- **exec():** Replaces the child process memory space and program code with a completely new executable file.

Fig 1.9: fork() and exec() Process Creation Flow



Q12. Define open source operating system. [2M]

(Source: Mid-1 Imp)

An open-source operating system is software released under an open-source license where anyone can freely inspect, modify, and redistribute the human-readable source code. Linux is the most prominent example, developed by Linus Torvalds and a worldwide community.

Q13. List the services provided by an Operating System. [2M]

(Source: Mid-1 Imp)

Key services provided to users and programs include: 1. Program Execution, 2. I/O Operations, 3. File System Manipulation, 4. Inter-process Communications, 5. Error Detection, 6. Resource Allocation, and 7. Protection and Security.

Q14. What is the purpose of system programs? [2M]

(Source: Mid-1 Imp)

System programs provide a convenient, user-friendly environment for program development and execution. They bridge user application needs with underlying system calls. Examples include compilers, assemblers, debuggers, file management utilities (copy, rm), and command interpreters (shells).

SECTION B: 5-MARK QUESTIONS (Detailed Conceptual Answers)

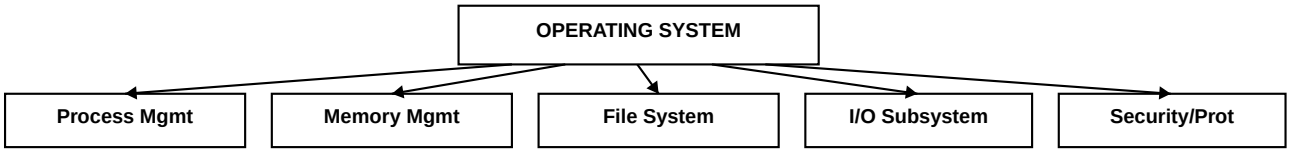
Q1. Explain in detail the functions of an operating system. [5M]

(Source: University Exam 2026 / Model Paper-I / Mid-1 Imp)

An operating system acts as a resource allocator and control program. Its essential functions include:

- **1. Process Management:** The OS controls the creation, scheduling, synchronization, communication, and termination of user and system processes. It allocates CPU time slices.
- **2. Main Memory Management:** Main memory is volatile and fast. The OS tracks which byte locations are occupied and by whom, dynamically allocates memory spaces, and deallocates them when processes finish.
- **3. File System Management:** The OS hides physical disk peculiarities and creates a logical view of files and directories. It handles creating, deleting, reading, writing, and directory searching.
- **4. Device (I/O) Management:** Manages I/O devices via a unified device-driver subsystem, including buffering (storing data in transit), caching (speed matching), and spooling.
- **5. Secondary Storage Management:** Hard disks and SSDs store data permanently. The OS manages free disk space, block allocation, and disk arm scheduling algorithms.
- **6. Protection & Security:** Ensures that system resources (memory, CPU, files) are accessed only by authorized processes and users.

Fig 1.10: Core Subsystem Functions of an Operating System



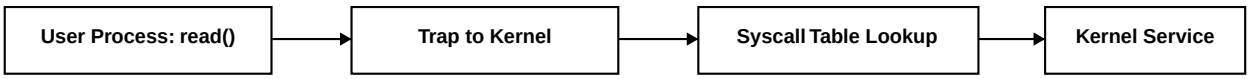
Q2. Describe the types of system calls with examples. [5M]

(Source: University Exam 2026 / Model Paper-II / Mid-1 Imp)

System calls are categorized into five major classes based on the nature of the service requested from the kernel:

Category	Purpose / Activity	UNIX / POSIX Example	Windows API Example
Process Control	Create, end, abort, load, wait, allocate memory	fork(), exit(), wait()	CreateProcess(), ExitProcess()
File Management	Create, delete, open, close, read, write file	open(), read(), write()	CreateFile(), ReadFile()
Device Management	Request, release, read, write physical devices	ioctl(), read(), write()	SetConsoleMode()
Information Maintenance	Get/set system time, process attributes, date	time(), getpid()	GetSystemTime(), GetCurrentProcessId()
Communication	Message passing or shared memory between processes	pipe(), shmget(), socket()	CreatePipe(), MapViewOfFile()

Fig 1.11: Step-by-Step System Call Execution Flow



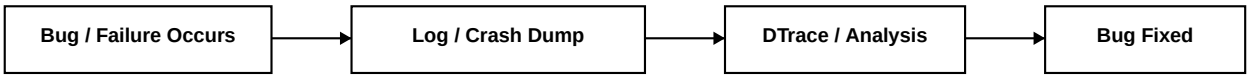
Q3. Discuss the importance of debugging in operating system development. [5M]

(Source: University Exam 2026 / Mid-1 Imp)

Operating system debugging is the activity of finding and fixing hardware and software errors (bugs) and tuning performance bottlenecks:

- **Failure Analysis:** When a user process fails, error info is written to a log file and memory is dumped into a core dump file. When the kernel itself fails, it causes a system crash, and memory is saved in a crash dump.
- **Performance Tuning:** Monitors bottlenecks using event tracing. All interesting events are logged with timestamps into trace files and analyzed to remove delays.
- **DTrace (Dynamic Tracing):** A dynamic instrumentation facility that inserts non-destructive probes into running user processes and kernel code without halting the entire OS, queried using the D programming language.

Fig 1.12: Operating System Debugging and Failure Analysis Cycle



Q4. Explain the difference between system programs and application programs. [5M]

(Source: Model Paper-I)

Software running on a computer is divided into system programs and application programs based on their functional proximity to the hardware:

Basis of Difference	System Programs	Application Programs
Definition	Programs that support the operation, management, and maintenance of the computer itself	Programs developed to solve specific user problems or perform personal tasks
Direct Interaction	Interacts closely with OS kernel and hardware via system calls	Interacts primarily with end users through user interfaces
Execution Mode	Runs with system privileges or interfaces directly with kernel	Runs strictly in restricted user mode
Examples	Compilers, linkers, shells (bash), disk formatters, device drivers	Web browsers, word processors, media players, database systems
Dependency	Essential for the functioning and usability of the OS	Optional; machine runs fine even if uninstalled

Q5. Explain the steps involved in building and booting an operating system. [5M]

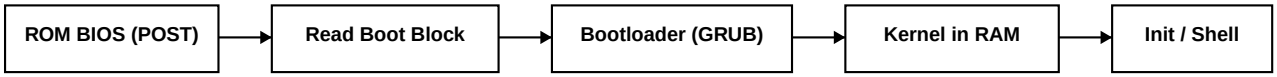
(Source: Model Paper-II)

Starting a computer by loading the kernel into physical RAM is called "booting":

- **1. Power-On Reset:** When powered on, CPU execution starts at a hardwired memory address in non-volatile ROM containing the initial Bootstrap Program (BIOS / UEFI).
- **2. POST (Power-On Self-Test):** Bootstrap runs hardware diagnostics to check RAM, keyboard, disk drives, and bus controllers.
- **3. Loading Boot Sector:** The simple bootstrap loader reads the Master Boot Record (MBR) or boot block (block 0 of disk) into memory.
- **4. Complex Bootloader (GRUB/Windows Boot Mgr):** The boot block loads the more powerful bootloader program from disk, which presents the OS selection menu.

• **5. Kernel Loading & Init:** The bootloader unpacks the kernel image into RAM, initializes CPU registers, memory tables, mounts root filesystem, and launches `init` (or `systemd`).

Fig 1.13: Five-Stage System Boot Sequence



Q6. Write the differences between monolithic kernel and microkernel. [5M]

(Source: Mid-1 Imp)

Kernel architecture determines how services are placed inside memory and protection domains:

Comparison Parameter	Monolithic Kernel	Microkernel Architecture
Kernel Size	Large, heavy kernel holding all OS services in single address space	Very small, minimalist kernel containing only essentials
Services Location	File systems, device drivers, memory, scheduling all inside kernel	Only IPC, basic memory paging, and CPU scheduling inside kernel
Execution Space	Everything runs in privileged kernel mode	Most services run as user-level daemons / servers
Performance	High performance due to direct fast function calls	Slower due to overhead of frequent message passing across modes
Reliability / Crash	If a single driver crashes, entire OS crashes (BSOD)	If a driver fails, it restarts safely without bringing down kernel
Examples	Traditional UNIX, Linux, MS-DOS, Windows 98	Mach, QNX, Minix, macOS (XNU hybrid)

Q7. What is operating system? Explain Computing Environments. [5M]

(Source: Mid-1 Imp)

A computing environment refers to the organizational setup of hardware and network components in which an operating system executes:

• **1. Traditional Computing:** Standalone desktop PCs or workstations connected to departmental network servers for file and print sharing.

• **2. Client-Server Computing:** Workstations (clients) send service requests over network to specialized servers: compute servers (database processing) or file servers (web content).

• **3. Peer-to-Peer (P2P) Computing:** All nodes are equal peers; every computer acts as both client and server, sharing files directly without central server bottleneck.

• **4. Web-Based & Cloud Computing:** Thin clients access computing infrastructure, platforms, and applications hosted on remote cloud data centers via web browsers.

Q8. Explain briefly User and Operating System Interface. [5M]

(Source: Mid-1 Imp)

Users interact with the operating system via two primary interface models:

• **Command Line Interface (CLI):** Textual command interpreter (shell) that reads direct text commands from keyboard (e.g., bash, PowerShell). Commands are built-in or executable file names. Fast, scriptable, and resource efficient.

• **Graphical User Interface (GUI):** Visual desktop metaphor with mouse, windows, menus, and icons invented at Xerox PARC. Highly user-friendly for non-programmers (e.g., Windows Desktop, macOS Aqua).

• **Touchscreen Interface:** Found on modern smartphones/tablets using gestures (swipes, pinches) on virtual touch keyboards.

Q9. Write short notes on Operating system Design & Implementation. [5M]

(Source: Mid-1 Imp)

Operating system design involves setting distinct design goals, separating mechanisms from policies, and selecting programming languages:

• **User Goals vs System Goals:** Users demand convenience, speed, safety, and reliability. System developers demand easy design, implementation, maintenance, and flexibility.

• **Mechanisms vs Policies:** A mechanism determines *how* to do something; a policy determines *what* will be done. Separating them is vital: policies change across installations (e.g., CPU priority), but mechanisms (timer interrupt switching) remain unchanged.

• **Implementation Language:** Early OS were written in assembly. Modern OS are written in higher-level languages (C, C++, Rust) for portability, compactness, and easier debugging.

Q10. Explain System Call Parameter Passing Methods. [5M]

(Source: Mid-1 Imp)

When an application calls a system call, it must pass parameters (e.g., file names, buffer addresses). Three standard techniques are used:

- **1. Pass in Registers:** Simplest method; parameters are loaded into CPU registers before executing trap. Limitation: Number of parameters is limited by available registers.
- **2. Block / Table in Memory:** Parameters are stored in a contiguous block in memory, and the starting memory address of the block is passed in a single register (used by Linux/Solaris).
- **3. Push onto Stack:** Parameters are pushed onto user stack by the calling program and popped off by the OS kernel routine. No limit on parameter count.

SECTION C: 10-MARK / LAQ QUESTIONS (Comprehensive University Answers)

Q1. Explain in detail the structure of an operating system (Simple, Layered, Microkernel, and Modular structures with neat sketches). [10M]

(Source: University Exam 2026 / Model Paper-I / Mid-1 Imp)

Operating system structure refers to the architectural organization and interconnection of kernel components. The four primary historical and modern architectures are:

1. Simple / Monolithic Structure (e.g., MS-DOS, Original UNIX)

In simple structures, the OS has no well-defined separation of levels. Applications can directly invoke hardware I/O routines. In original UNIX, everything (files, memory, scheduling) was bundled into a single monolithic kernel below system calls.

- **Advantage:** Very fast performance due to few software layers and minimal message passing.
- **Disadvantage:** Vulnerable to crashes; if any user program or driver fails, the entire system crashes. Hard to maintain.

2. Layered Structure

The operating system is broken into a hierarchy of layers (Layer 0 to Layer N). Layer 0 is hardware; Layer N is user interface. Each layer uses only lower-level services.

- **Advantage:** Simplifies debugging and verification because lower layers are thoroughly tested before higher layers are added.
- **Disadvantage:** Difficult to define layer boundaries properly; efficiency is lower due to traversal of multiple layers.

3. Microkernel Structure (e.g., Mach, QNX, Minix)

Removes all non-essential components from the kernel and runs them as user-level system programs. The kernel retains only three critical functions: minimal process management, minimal memory addressing, and inter-process communication (IPC).

- **Advantage:** Extremely high security and reliability. If a driver or file system crashes, it does not kill the kernel.
- **Disadvantage:** Performance penalty caused by frequent context switching and IPC message copying.

4. Modular / Dynamically Loadable Structure (e.g., Solaris, Linux LKM)

Considered the best modern design. The core kernel provides fundamental services, and additional features (device drivers, file systems, network protocols) are loaded dynamically into kernel space during boot time or on-demand at runtime.

Diagram 1 (Standard Exam Diagram): Operating System Structural Models

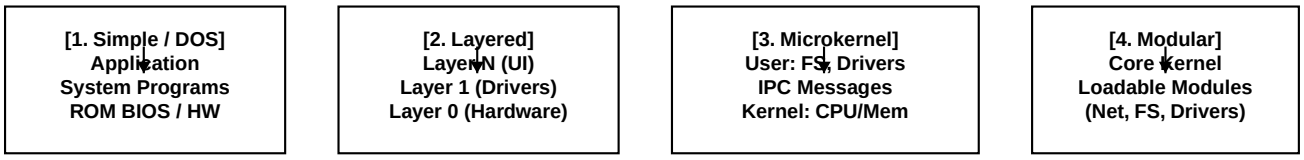


Diagram 2 (Concept Understanding Flow): Communication & Boundary in Microkernel vs Monolithic



Q2. Explain about the dual mode operation in OS with a neat block diagram and transition mechanism. [10M]

(Source: Mid-1 Imp)

Dual-mode operation ensures hardware-level protection by separating program execution into two distinct operating modes:

- **User Mode (Mode Bit = 1):** When computer hardware executes code on behalf of a user application, the mode bit is set to 1. User programs cannot directly execute privileged instructions (such as disabling interrupts or modifying page tables).
- **Kernel Mode (Mode Bit = 0, also Supervisor / System / Privileged Mode):** When the operating system gains control of the computer, the hardware flips the mode bit to 0. All CPU instructions and all memory areas are accessible.
- **Mode Transition Mechanism:** Whenever an interrupt, exception (divide by zero), or system call occurs, hardware automatically switches the mode bit to 0 and jumps to the kernel Interrupt Service Routine (ISR). When finished, the kernel executes a privileged return instruction that resets the mode bit to 1.
- **Timer Hardware Control:** To prevent infinite loops from locking the CPU in user mode, a hardware timer decrements every clock cycle. When it reaches 0, a timer interrupt transfers control back to kernel mode.

Diagram 1 (Standard Exam Diagram): User to Kernel Mode Transition

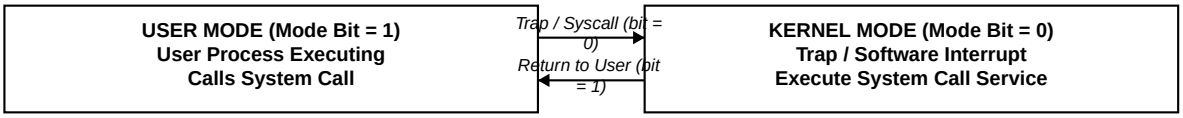
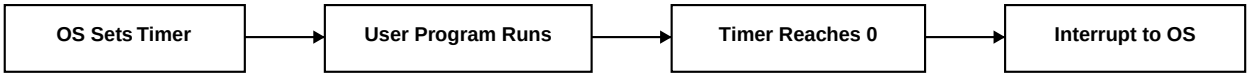


Diagram 2 (Concept Understanding Flow): Hardware Timer Protection Cycle



UNIT - II: PROCESSES, THREADS & CPU SCHEDULING

SECTION A: 2-MARK QUESTIONS (Short, Direct Answers)

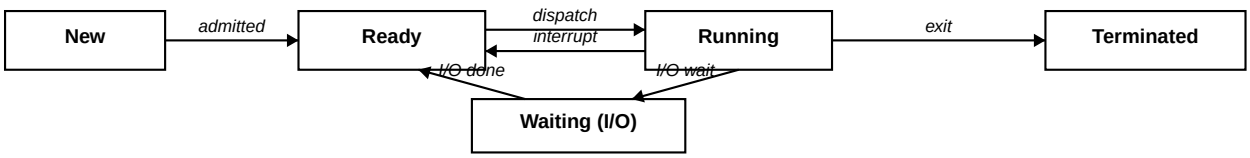
Q1. What are the different states of a process? [2M]

(Source: University Exam 2026 / Mid-1 Imp)

A process changes state as it executes. The five standard states are:

- **1. New:** The process is currently being created.
- **2. Ready:** The process is loaded into RAM waiting to be assigned to a CPU core.
- **3. Running:** Instructions of the process are being executed on the CPU.
- **4. Waiting (Blocked):** The process is waiting for an I/O event or signal.
- **5. Terminated:** The process has completed execution.

Fig 2.1: 5-State Process Model



Q2. Differentiate preemptive and non-preemptive scheduling. [2M]

(Source: University Exam 2026 / Model Paper-I)

In non-preemptive scheduling, once the CPU is allocated to a process, it holds the CPU until it terminates or switches to waiting state (e.g., standard FCFS). In preemptive scheduling, the OS can forcefully interrupt and take away the CPU from a running process when a higher-priority process arrives or a time-slice expires (e.g., Round Robin, SRTF).

Q3. What is a Process Control Block (PCB)? / What does PCB contain? [2M]

(Source: Model Paper-I / Mid-1 Imp)

A Process Control Block (PCB, also called Task Control Block) is a kernel data structure that stores all critical administrative information required to manage a specific process.

- **Contents:** Process ID (PID), Process State, Program Counter (PC), CPU Registers, CPU Scheduling Info (Priority), Memory Limits, and List of Open Files.

Fig 2.2: Process Control Block (PCB) Layout

Process ID & State
Program Counter (PC)
CPU Registers & Stack Pointer
Memory Limits & Page Table
List of Open I/O Devices

Q4. List any two CPU scheduling algorithms. [2M]

(Source: Model Paper-I)

Two fundamental CPU scheduling algorithms are:

- **1. First-Come, First-Served (FCFS):** Non-preemptive; assigns CPU in order of arrival.
- **2. Shortest Job First (SJF):** Assigns CPU to the process with smallest burst time.

Q5. Differentiate between process and thread. [2M]

(Source: Model Paper-II)

A process is an independent program in execution with its own separate virtual address space. A thread is a lightweight unit of execution within a process that shares memory and resources with other peer threads.

Comparison Point	Process	Thread
Resource Allocation	Heavyweight; allocates dedicated memory space	Lightweight; shares address space of parent process
Context Switch	Slow and costly; involves TLB flushing	Fast and low-overhead; minimal register saving
Communication	Requires IPC (pipes, shared memory, sockets)	Direct communication via shared process variables

Q6. What is the role of a dispatcher in CPU scheduling? [2M]

(Source: Model Paper-I / Model Paper-II)

The dispatcher is the OS kernel module that gives physical control of the CPU to the process selected by the short-term scheduler.

- **Three Main Duties:** 1. Switching context; 2. Switching to user mode; 3. Jumping to the proper instruction location in the user program to restart it.
- **Dispatch Latency:** The time taken by the dispatcher to stop one process and start another.

Q7. Define process. [2M]

(Source: Mid-1 Imp)

A process is defined as a program in execution. Unlike a passive program stored as an executable file on disk, a process is an active entity with a Program Counter, processor registers, stack (temporary data), data section (global variables), and heap (dynamically allocated memory).

Q8. Define schedulers. [2M]

(Source: Mid-1 Imp)

Schedulers are specialized system software routines that manage process queue transitions. They determine which processes are admitted into the system, which are swapped to disk, and which process gets the CPU core.

Q9. What are the types of scheduler? [2M]

(Source: Mid-1 Imp)

The three primary types of schedulers are:

- **Long-Term Scheduler (Job Scheduler):** Selects jobs from disk spool and loads them into memory; controls the degree of multiprogramming.
- **Short-Term Scheduler (CPU Scheduler):** Selects from among ready processes in memory and allocates CPU core; runs very frequently (milliseconds).
- **Medium-Term Scheduler (Swapper):** Temporarily removes processes from RAM to disk swap space to reduce memory contention.

Q10. Define Thread. [2M]

(Source: Mid-1 Imp)

A thread is a basic unit of CPU utilization. It comprises a Thread ID, a Program Counter, a register set, and a stack. It shares its code section, data section, and OS resources (such as open files) with other threads belonging to the same process.

Q11. What is multi processor? [2M]

(Source: Mid-1 Imp)

A multiprocessor system (parallel system) contains two or more physical CPUs or cores that share computer bus, clock, memory, and peripheral devices. It increases system throughput and reliability through graceful degradation.

Q12. Define Context Switching. [2M]

(Source: Mid-1 Imp)

Context switching is the mechanism of saving the current state (context) of a running process in its PCB and loading the saved context of another ready process into CPU registers so that execution can switch seamlessly.

Q13. List the Thread libraries. [2M]

(Source: Mid-1 Imp)

A thread library provides programmers with an API for creating and managing threads. The three main thread libraries are: 1. POSIX Pthreads (IEEE standard), 2. Windows Threads (Win32), and 3. Java Threads (JVM managed).

SECTION B: 5-MARK QUESTIONS (Detailed Conceptual Answers)

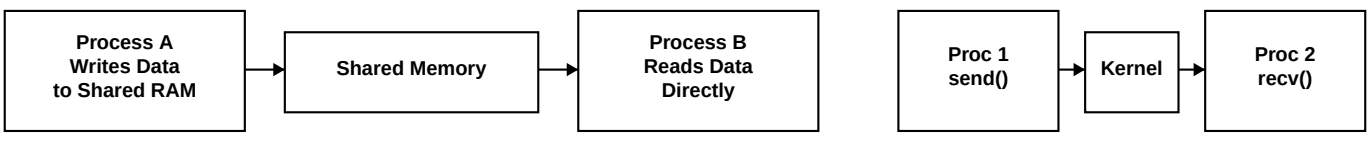
Q1. Describe message-passing and shared-memory models of IPC. [5M]

(Source: University Exam 2026 / Mid-1 Imp)

Cooperating processes communicate and synchronize actions through Inter-Process Communication (IPC). The two fundamental IPC architectural models are:

- **Shared-Memory Model:** A region of shared memory is mapped into the virtual address space of communicating processes. Once established, processes exchange data by normal reading and writing to this shared memory space without kernel intervention. It offers very fast data transfer but requires explicit process synchronization (semaphores) to avoid race conditions.
- **Message-Passing Model:** Processes exchange messages using two primitive kernel system calls: `send(message)` and `receive(message)`. Useful in distributed systems and network environments. It is easier to program and safer, but slower due to frequent kernel system call traps and message copying.

Fig 2.3: Comparison of Shared-Memory vs Message-Passing IPC Models



Q2. Discuss multithreading models in detail. [5M]

(Source: University Exam 2026 / Model Paper-II / Mid-1 Imp)

Threads exist at two levels: User Threads (managed by user-level library without kernel knowledge) and Kernel Threads (supported and managed directly by the OS kernel). The relationship between them is established via three multithreading models:

Model	Mapping Scheme	Pros & Cons	Examples
Many-to-One	Many user threads mapped to a single kernel thread	Fast management; BUT if one thread blocks on I/O, entire process blocks. No true multicore parallelism.	Green Threads (Solaris), GNU Portable Threads
One-to-One	Each user thread mapped to a separate kernel thread	True multicore parallelism; one thread blocking does not block others; BUT creating thread adds kernel overhead.	Linux, Windows (NT, 10, 11), macOS
Many-to-Many	Many user threads multiplexed to smaller or equal kernel threads	Best balance; no restriction on user thread count; true multicore execution without blocking.	Solaris with ThreadFly, IRIX, HP-UX

Q3. Explain about process schedulers (Long-term, Short-term, Medium-term). [5M]

(Source: Mid-1 Imp)

Processes migrate among various scheduling queues throughout their life cycle. The operating system utilizes three levels of schedulers to coordinate this queue migration:

Scheduler	Other Name	Frequency of Execution	Key Responsibility
Long-Term	Job Scheduler	Infrequent (seconds/minutes)	Controls degree of multiprogramming; maintains balanced mix of I/O-bound and CPU-bound jobs.
Short-Term	CPU Scheduler	Very frequent (milliseconds)	Selects next ready process to execute on CPU; must be extremely fast to minimize system overhead.
Medium-Term	Swapper	Intermediate	Swaps partially executed blocked processes out of RAM to disk swap space to free memory; swaps them back later.

Q4. Explain about Scheduling Criteria. [5M]

(Source: Mid-1 Imp)

Different CPU scheduling algorithms have different properties. To evaluate and compare them, operating systems rely on five standardized scheduling criteria:

- **1. CPU Utilization:** The percentage of time that the CPU is actively performing useful work (target: 40% to 90%). Maximize.
- **2. Throughput:** The number of complete processes executed per unit time (e.g., 10 processes/min). Maximize.
- **3. Turnaround Time (TAT):** The total elapsed time interval from process submission to process completion: TAT = Completion Time - Arrival Time. Minimize.
- **4. Waiting Time (WT):** The total sum of time periods spent by a process waiting inside the Ready Queue: WT = Turnaround Time - Burst Time. Minimize.
- **5. Response Time:** The time taken from submission until the first response is produced (crucial in interactive systems). Minimize.

Q5. Discuss about threading issues. [5M]

(Source: Mid-1 Imp)

Developing multithreaded systems introduces unique design and semantic complications:

- **1. fork() and exec() Semantics:** If one thread calls `fork()`, does the new process duplicate all threads or only the calling thread? (UNIX provides both flavors: `fork()` for all, and `fork1()` for single thread).
- **2. Thread Cancellation:** Terminating a thread before it finishes. Done asynchronously (target thread stopped immediately, may leave shared data corrupt) or deferred (target thread periodically checks cancellation points safely).
- **3. Signal Handling:** In UNIX, signals notify processes of events (e.g., Ctrl+C). In multithreading, where should the signal be delivered? To the thread caused it, to all threads, or to a dedicated thread?
- **4. Thread Pools:** Creating unlimited threads exhausts memory. A thread pool maintains a fixed number of standing worker threads waiting for tasks.

Q6. Write about Operations on Processes (Creation and Termination). [5M]

(Source: Mid-1 Imp)

Operating systems provide mechanisms for process creation and process termination:

- **Process Creation:** A parent process creates child processes, which can in turn create children, forming a process hierarchy tree (e.g., `init` / `systemd` is the root). In UNIX, `fork()` creates a clone, and `exec()` overlays a new program.
- **Resource Sharing Options:** 1. Parent and children share all resources; 2. Children share a subset of parent resources; 3. Child receives separate resources.
- **Process Termination:** A process terminates by executing `exit()` system call. Its OS resources (RAM, files) are deallocated. A terminated process remains in process table as a "Zombie" until parent calls `wait()`. If parent exits without calling `wait()`, children become "Orphan" processes (adopted by `systemd`).

SECTION C: 10-MARK / LAQ QUESTIONS (Comprehensive University Answers)

Q1. Explain the process concept with a neat diagram of process states and explain Process Control Block (PCB) in detail. [10M]

(Source: Model Paper-II / Mid-1 Imp)

A process is the central abstraction in any modern multitasking operating system. It represents a program in active execution.

1. Memory Representation of a Process

A process image in physical memory is divided into four distinct sections:

- **Text Section:** Stores compiled machine instructions and executable code (read-only).
- **Data Section:** Contains global and static variables initialized before runtime.
- **Heap Section:** Memory dynamically allocated during runtime (via `malloc()` in C or `new` in Java/C++).
- **Stack Section:** Contains temporary execution data such as function parameters, return addresses, and local variables. Grows downward toward the heap.

2. Detailed Process State Lifecycle

A process transitions through five primary lifecycle states: New (creation) -> Ready (queued in RAM) -> Running (CPU execution) -> Waiting (blocked on I/O) -> Terminated (resources freed). If a running process exhausts its time slice, it is preempted back to the Ready queue.

3. Process Control Block (PCB) Structure & Components

The operating system represents each process using a Process Control Block (PCB). When a context switch occurs, the dispatcher saves the CPU registers and Program Counter of the current process into its PCB, and restores the values from the new process's PCB.

Diagram 1 (Standard Exam Diagram): Detailed Process State Transition Architecture

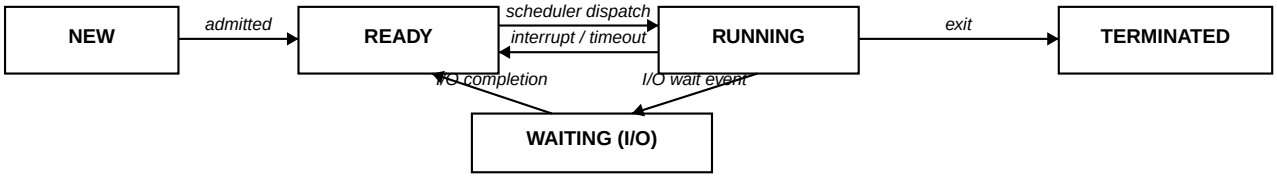
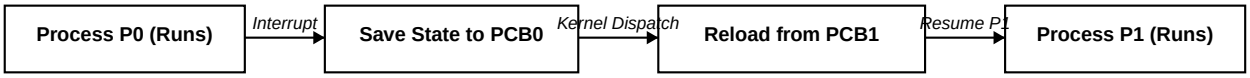


Diagram 2 (Concept Understanding Flow): Context Switch Execution between Process P0 and P1



SECTION D: NUMERICAL PROBLEMS (Step-by-Step Solved Calculations)

Q1. Given the processes below, schedule them using FCFS. Calculate: (i) Completion Time, (ii) Average Waiting Time, (iii) Average Turnaround Time. [5M]

(Source: University Exam 2026)

Step 1 – Given Data:

Process ID	Arrival Time (AT)	Burst Time (BT)
P1	1	4
P2	3	5
P3	5	2
P4	7	3

Step 2 – Required Answer:

Calculate: (i) Completion Time (CT) for each process; (ii) Turnaround Time (TAT = CT - AT); (iii) Waiting Time (WT = TAT - BT); (iv) Average Waiting Time; (v) Average Turnaround Time.

Step 3 – Gantt Chart Execution Sequence:
At t = 0, CPU is idle. At t = 1, P1 arrives and runs until t = 5. At t = 5, P2 runs until t = 10. At t = 10, P3 runs until t = 12. At t = 12, P4 runs until t = 15.

Gantt Chart Representation for FCFS Scheduling

Idle	P1 (1 to 5)	P2 (5 to 10)	P3 (10 to 12)	P4 (12 to 15)

Step 4 & 5 – Formulas & Calculation Table:
Formulas: Turnaround Time (TAT) = CT - AT; Waiting Time (WT) = TAT - BT.

Process	AT	BT	Completion Time (CT)	TAT = CT - AT	WT = TAT - BT
P1	1	4	5	5 - 1 = 4	4 - 4 = 0
P2	3	5	10	10 - 3 = 7	7 - 5 = 2
P3	5	2	12	12 - 5 = 7	7 - 2 = 5
P4	7	3	15	15 - 7 = 8	8 - 3 = 5

Step 6 – Averages Calculation:
Total Turnaround Time = 4 + 7 + 7 + 8 = 26

Average Turnaround Time = 26 / 4 = 6.5 time units

Total Waiting Time = 0 + 2 + 5 + 5 = 12

Average Waiting Time = 12 / 4 = 3.0 time units

[FINAL ANSWER: FCFS CPU Scheduling Metrics]
Completion Times: P1 = 5, P2 = 10, P3 = 12, P4 = 15
Average Turnaround Time (TAT) = 6.5 time units
Average Waiting Time (WT) = 3.0 time units

Q2. Consider 3 processes P1, P2 and P3 requiring 5, 7 and 4 time units and arrive at time 0, 1 and 3. Draw Gantt chart and average waiting time for: (i) Round Robin (quantum = 2); (ii) FCFS. [5M]
(Source: Mid-1 Imp)

(i) Round Robin Scheduling (Quantum = 2):
At t=0: P1 arrives (rem=5). P1 runs [0,2], rem=3. At t=1, P2 arrives (rem=7). At t=2, P1 moved to back of queue. Ready queue: P2, P1.

P2 runs [2,4], rem=5. At t=3, P3 arrives (rem=4). Ready queue: P1, P3, P2.

P1 runs [4,6], rem=1. Queue: P3, P2, P1.

P3 runs [6,8], rem=2. Queue: P2, P1, P3.

P2 runs [8,10], rem=3. Queue: P1, P3, P2.

P1 runs [10,11], rem=0 -> P1 Completes at t = 11! Queue: P3, P2.

P3 runs [11,13], rem=0 -> P3 Completes at t = 13! Queue: P2.

P2 runs [13,16], rem=0 -> P2 Completes at t = 16!

Process	AT	BT	Completion Time (CT)	TAT = CT - AT	WT = TAT - BT
P1	0	5	11	11 - 0 = 11	11 - 5 = 6
P2	1	7	16	16 - 1 = 15	15 - 7 = 8
P3	3	4	13	13 - 3 = 10	10 - 4 = 6

Average Waiting Time (RR) = (6 + 8 + 6) / 3 = 20 / 3 = 6.67 time units.

(ii) FCFS Scheduling:
P1 runs [0, 5], CT=5. P2 runs [5, 12], CT=12. P3 runs [12, 16], CT=16.

Process	AT	BT	Completion Time (CT)	TAT = CT - AT	WT = TAT - BT
P1	0	5	5	5 - 0 = 5	5 - 5 = 0
P2	1	7	12	12 - 1 = 11	11 - 7 = 4
P3	3	4	16	16 - 3 = 13	13 - 4 = 9

Average Waiting Time (FCFS) = (0 + 4 + 9) / 3 = 13 / 3 = 4.33 time units.

[FINAL ANSWER: Comparison of RR vs FCFS Results]
Round Robin (Q=2): Avg WT = 6.67 time units | Avg TAT = 12.0 time units
FCFS: Avg WT = 4.33 time units | Avg TAT = 9.67 time units

UNIT - III: SYNCHRONIZATION TOOLS & DEADLOCKS

SECTION A: 2-MARK QUESTIONS (Short, Direct Answers)

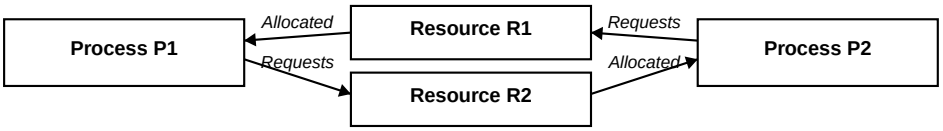
Q1. Define busy waiting. [2M]
(Source: University Exam 2026 / Mid-1 Imp)
Busy waiting is a condition where a process repeatedly checks a condition inside a tight loop (e.g., `while (flag != true);`) while waiting for access to a critical section. It wastes valuable CPU execution cycles that other active ready processes could use.

Q2. What is starvation? [2M]
(Source: University Exam 2026)
Starvation (indefinite blocking) is a situation where an execution-ready process is perpetually denied access to a necessary system resource or CPU core because other higher-priority processes continually preempt it.

Q3. State Peterson's solution for the critical section problem. [2M]
(Source: Model Paper-I)
Peterson's algorithm is a software-based solution for two processes (P_0 and P_1) to achieve mutual exclusion without hardware support. It relies on two shared variables: `int turn;` (whose turn it is) and `boolean flag[2];` (indicating desire to enter the critical section).

Q4. Define deadlock in operating systems. [2M]
(Source: Model Paper-I / Mid-1 Imp)
A deadlock is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process in the same set, resulting in none of them ever progressing.

Fig 3.1: Circular Deadlock Dependency



Q5. What is a semaphore? / Define semaphores. [2M]
(Source: Model Paper-II / Mid-1 Imp)
A semaphore S is an integer synchronization variable accessed only through two atomic, indivisible operations: `wait(S)` (or `P()`, which decrements S) and `signal(S)` (or `V()`, which increments S). Used to control access to shared concurrent resources.

Q6. Mention two conditions necessary for deadlock to occur. [2M]
(Source: Model Paper-II)
Two of the four necessary Coffman conditions are:

- **Mutual Exclusion:** At least one resource must be held in a non-shareable mode (only one process at a time).
- **Hold and Wait:** A process must be holding at least one resource and simultaneously waiting to acquire additional resources held by other processes.

Q7. Define race condition. [2M]
(Source: Mid-1 Imp)
A race condition is an undesirable situation where multiple processes concurrently read and write to shared data, and the final outcome of the execution depends entirely on the unpredictable order in which instructions are scheduled.

Q8. What are the requirements that a solution to the critical section problem must satisfy? [2M]
(Source: Mid-1 Imp)
Any correct solution must satisfy three strict requirements:

- **1. Mutual Exclusion:** If process P_i is executing in its critical section, no other processes can be executing in their critical sections.
- **2. Progress:** If no process is executing in its critical section and some processes wish to enter, only those processes not in their remainder section can participate in deciding who enters next.
- **3. Bounded Waiting:** There must be a bound on the number of times other processes are allowed to enter their critical sections after a process has made a request.

Q9. Name the classic problems of synchronization. [2M]
(Source: Mid-1 Imp)
The three classic synchronization benchmark problems are: 1. The Bounded-Buffer (Producer-Consumer) Problem; 2. The Readers-Writers Problem; 3. The Dining-Philosophers Problem.

Q10. What is the critical section problem? [2M]
(Source: Mid-1 Imp)
The critical section is a segment of code in which a process accesses shared resources (common variables, tables, or files). The critical section problem is to design a protocol that cooperating processes can use to synchronize their actions so that no two processes execute their critical sections concurrently.

Q11. Define spinlock. [2M]
(Source: Mid-1 Imp)
A spinlock is a mutex lock where a process waiting to acquire the lock spins continuously in a loop checking the lock variable. While it causes busy waiting, spinlocks are very useful on multiprocessor architectures when locks are expected to be held for short durations (saving context switch time).

Q12. What is Process Synchronization? [2M]
(Source: Mid-1 Imp)
Process synchronization is the coordination of concurrent execution of multiple cooperating processes to ensure ordered access to shared data and prevent data inconsistency and race conditions.

Q13. List the four necessary conditions for a deadlock to occur. [2M]
(Source: Mid-2 QB)
Deadlock occurs if and only if all four Coffman conditions hold simultaneously: 1. Mutual Exclusion, 2. Hold and Wait, 3. No Preemption (resources cannot be forcibly confiscated), and 4. Circular Wait (a circular chain of processes exists where each waits for a resource held by the next).

Q14. Differentiate between deadlock prevention and deadlock avoidance. [2M]
(Source: Mid-2 QB)
Deadlock prevention imposes static protocol rules that ensure at least one of the four necessary conditions can never hold. Deadlock avoidance dynamically examines every resource request using knowledge of future resource demands (e.g., Banker's algorithm) to ensure the system remains in a safe state.

Q15. What is a wait-for graph and resource-allocation graph? [2M]
(Source: Mid-2 QB)
A Resource-Allocation Graph (RAG) contains processes (circles) and resources (rectangles). For single-instance resources, a cycle indicates a deadlock. A Wait-For Graph is a collapsed version containing only processes: directed edge $P_i \rightarrow P_j$ means P_i is waiting for P_j to release a resource. A cycle in a wait-for graph strictly implies deadlock.

Q16. What is a safe state in deadlock avoidance? [2M]
(Source: Mid-2 QB)
A state is safe if the system can allocate resources to each process (up to its maximum need) in some order and still avoid a deadlock. That is, a safe state guarantees the existence of at least one "safe sequence" $\langle P_1, P_2, \dots, P_n \rangle$ of process completion.

SECTION B: 5-MARK QUESTIONS (Detailed Conceptual Answers)

Q1. Explain Readers–Writers Problem using semaphores. [5M]

(Source: University Exam 2026)

The Readers-Writers problem models access to a shared database. Multiple "readers" may read concurrently without conflict, but a "writer" requires exclusive access (no other reader or writer allowed).

- **Synchronization Variables:** `semaphore rw_mutex = 1;` (mutual exclusion for writers), `semaphore mutex = 1;` (protects `read_count`), `int read_count = 0;` (number of active readers).
- **Writer Process Structure:** Executes `wait(rw_mutex);` -> writes data -> executes `signal(rw_mutex);`.
- **Reader Process Structure:** Executes `wait(mutex); read_count++; if (read_count == 1) wait(rw_mutex); signal(mutex);` -> reads data -> `wait(mutex); read_count--; if (read_count == 0) signal(rw_mutex); signal(mutex);`. The first reader locks out writers, and the last reader releases writers.

Q2. Discuss the necessary conditions for deadlock with examples. [5M]

(Source: University Exam 2026 / Model Paper-I / Model Paper-II / Mid-2 QB)

A deadlock condition arises if and only if the four Coffman conditions occur simultaneously:

Condition	Technical Meaning	Real-World System Example
1. Mutual Exclusion	Resource cannot be shared; only one process holds it	A physical printer or write lock on a database record
2. Hold and Wait	Process holds allocated resource while waiting for another	Process holds tape drive and requests disk drive
3. No Preemption	Resources cannot be seized; released voluntarily by holder	CPU cannot confiscate tape drive until job finishes
4. Circular Wait	Chain $\{P_0, P_1, \dots, P_n\}$ where P_0 waits for P_1, P_n waits for P_0	Four cars arrive at a four-way crossing, each blocking the next

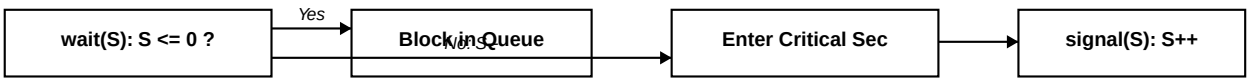
Q3. Define semaphores and explain how they are used in process synchronization. [5M]

(Source: Model Paper-I / Mid-1 Imp)

A semaphore is an integer variable accessed only through atomic `wait()` and `signal()` operations. Operating systems support two types of semaphores:

- **Binary Semaphore (Mutex Lock):** Value ranges only between 0 and 1. Initialized to 1. Used to enforce mutual exclusion across critical sections.
- **Counting Semaphore:** Value can range over an unrestricted integer domain. Initialized to N (the number of available resource instances). Each `wait()` decrements the count; when count drops below 0, processes block until a resource is returned via `signal()`.

Fig 3.2: Semaphore wait() and signal() Logic Flow



Q4. Write short notes on Peterson's solution. [5M]

(Source: Model Paper-I)

Peterson's algorithm is a software solution for two processes (P_0 and P_1) to achieve mutual exclusion:

- **Data Structures:** `boolean flag[2];` (initially false) and `int turn;`. `flag[i] = true` means process i wants to enter.
- **Entry Section for Process Pi:** `flag[i] = true; turn = j; while (flag[j] && turn == j);` (wait until other process is not interested or gives up turn).
- **Exit Section for Process Pi:** `flag[i] = false;` (releases interest).
- **Correctness Verification:** 1. Mutual exclusion holds because `turn` can only be 0 or 1; 2. Progress and Bounded Waiting are satisfied.

Q5. What is Semaphore? Explain producer consumer problem using semaphore. [5M]

(Source: Mid-1 Imp)

The Bounded-Buffer Producer-Consumer problem synchronizes two processes sharing a buffer of size N:

- **Semaphores Used:** `semaphore mutex = 1;` (protects buffer manipulation), `semaphore empty = N;` (counts empty buffer slots), `semaphore full = 0;` (counts full slots).
- **Producer Code:** Produces item -> `wait(empty); wait(mutex);` -> add item to buffer -> `signal(mutex); signal(full);`.

• **Consumer Code:** `wait(full); wait(mutex);` -> remove item from buffer -> `signal(mutex); signal(empty);` -> consumes item.

Q6. What is Monitor? Explain with an example using monitor. [5M]
(Source: Mid-1 Imp)
A monitor is a high-level programming language synchronization construct that encapsulates shared data variables and the procedures that operate on them. Only one process can be active inside the monitor at any given time, automatically guaranteeing mutual exclusion without explicit semaphore programming.

• **Condition Variables:** To synchronize actions within a monitor, condition variables (e.g., `condition x;`) provide two methods: `x.wait()` (suspends calling process) and `x.signal()` (resumes exactly one suspended process).

Q7. Explain deadlock prevention and discuss the methods used to prevent deadlocks. [5M]
(Source: Mid-2 QB)
Deadlock prevention ensures that at least one of the four Coffman conditions is eliminated:

• **1. Eliminating Mutual Exclusion:** Make resources shareable (e.g., read-only files). However, intrinsic devices like printers cannot be shared.

• **2. Eliminating Hold and Wait:** Protocol requires a process to request and receive all resources before execution begins, or release current resources before requesting new ones. Drawback: Low resource utilization and starvation.

• **3. Eliminating No Preemption:** If a process holding resources requests another that cannot be allocated, all its held resources are preempted and returned to the pool.

• **4. Eliminating Circular Wait:** Impose a total ordering of all resource types ($F: R \rightarrow N$). A process can only request resources in strictly increasing order of enumeration ($F(R_i) < F(R_j)$). Highly practical.

Q8. Discuss the methods of recovery from deadlock, including process termination and resource preemption. [5M]
(Source: Mid-2 QB)
Once a deadlock detection algorithm identifies a deadlock, the OS breaks the deadlock using two strategies:

• **1. Process Termination:** Option A: Abort all deadlocked processes (clears deadlock immediately, but expensive since computations are lost); Option B: Abort one process at a time until the deadlock cycle is broken (requires re-running detection algorithm after each termination).

• **2. Resource Preemption:** Progressively preempt resources from processes and allocate them to other deadlocked processes. Three issues must be handled: 1. Selecting a victim (minimize preemption cost); 2. Rollback (roll process back to a safe checkpoint state); 3. Starvation (ensure the same process is not always chosen as victim).

SECTION C: 10-MARK / LAQ QUESTIONS (Comprehensive University Answers)

Q1. Explain how Dining Philosophers Problem can be solved using semaphores in detail. [10M]
(Source: University Exam 2026 / Model Paper-I / Model Paper-II / Mid-1 Imp)
The Dining Philosophers problem is a classic synchronization problem formulated by Edsger Dijkstra. Five philosophers sit around a circular table with five chopsticks and a bowl of rice. A philosopher spends life thinking and eating. To eat, a philosopher requires both the left and right chopsticks.

1. Naive Semaphore Representation & Deadlock Problem
Represent each chopstick as a binary semaphore: `semaphore chopstick[5] = {1, 1, 1, 1, 1};`. A philosopher i attempts:

• **Execution Code:** `wait(chopstick[i]); wait(chopstick[(i+1)%5]); eat(); signal(chopstick[i]); signal(chopstick[(i+1)%5]);`.

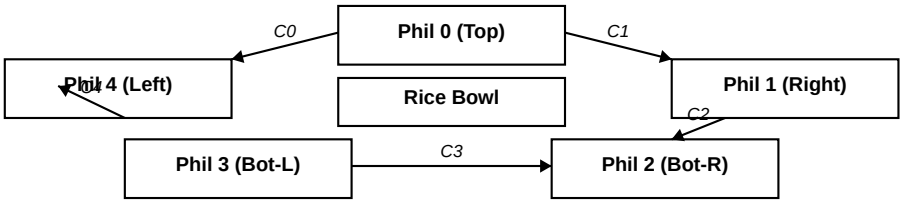
• **Deadlock Scenario:** If all five philosophers sit simultaneously and pick up their left chopstick, every chopstick semaphore drops to 0. When they attempt to grab their right chopstick, all five block indefinitely — causing total deadlock!

2. Deadlock-Free Solutions

• **Solution A (Asymmetric Solution):** An odd philosopher (P_1, P_3) picks up left chopstick first, then right. An even philosopher (P_0, P_2, P_4) picks up right chopstick first, then left. This eliminates circular wait.

• **Solution B (State-based Semaphore Solution):** Allow a philosopher to pick up chopsticks only if both chopsticks are simultaneously available. A philosopher transitions between THINKING, HUNGRY, and EATING states under mutex protection.

Diagram 1 (Standard Exam Diagram): Dining Philosophers Table Configuration



Q2. Explain deadlock avoidance with an example of Banker's Algorithm in detail (Safety & Resource-Request Algorithms). [10M]

(Source: University Exam 2026 / Mid-2 QB)

Banker's Algorithm, developed by Dijkstra, avoids deadlocks by dynamically ensuring the system never enters an unsafe state. When a process requests resources, the OS simulates allocation to verify that a safe sequence still exists before granting the request.

1. Data Structures Used

- **Available[m]:** Vector of length m. If `Available[j] = k`, k instances of resource R_j are available.
- **Max[n][m]:** $n \times m$ matrix. `Max[i][j]` defines max demand of process P_i for resource R_j.
- **Allocation[n][m]:** $n \times m$ matrix. `Allocation[i][j]` defines instances of R_j currently allocated to P_i.
- **Need[n][m]:** Remaining need: `Need[i][j] = Max[i][j] - Allocation[i][j]`.

2. Safety Algorithm

- **Step 1:** Initialize `Work = Available` and `Finish[i] = false` for all $i = 0, 1, \dots, n-1$.
- **Step 2:** Find an index i such that `Finish[i] == false` and `Need[i] <= Work`. If no such i exists, go to Step 4.
- **Step 3:** `Work = Work + Allocation[i]`; `Finish[i] = true`; go back to Step 2.
- **Step 4:** If `Finish[i] == true` for all i, system is in a SAFE state; otherwise UNSAFE.

3. Resource-Request Algorithm for Process Pi

When process P_i issues request `Request[i]`: 1. If `Request[i] <= Need[i]`, proceed; else error. 2. If `Request[i] <= Available`, proceed; else P_i must wait. 3. Pretend to allocate: `Available -= Request[i]; Allocation[i] += Request[i]; Need[i] -= Request[i];`. Run Safety algorithm. If safe, grant resources; if unsafe, rollback and make P_i wait.

SECTION D: NUMERICAL PROBLEMS (Step-by-Step Solved Calculations)

Q1. Consider the following snapshot and find out the possible resource allocation sequence with the help of deadlock detection algorithm: Processes P0-P4, Resources A, B, C. [5M]

(Source: OS Units 3,4,5 Problems Bank)

Step 1 – Given Data:

Process	Allocation (A B C)	Request (A B C)	Available (A B C)
P0	0 1 0	0 0 0	0 0 0
P1	2 0 0	2 0 2	
P2	3 0 3	0 0 0	
P3	2 1 1	1 0 0	
P4	0 0 2	0 0 2	

Step 2 – Execution Trace of Deadlock Detection Algorithm:

Initialize: Work = Available = [0, 0, 0]. Finish = [false, false, false, false, false].

1. Check P0: Request [0, 0, 0] <= Work [0, 0, 0]. P0 can finish! Work = [0, 0, 0] + [0, 1, 0] = [0, 1, 0]. Finish[P0] = true.

2. Check P2: Request [0, 0, 0] <= Work [0, 1, 0]. P2 can finish! Work = [0, 1, 0] + [3, 0, 3] = [3, 1, 3]. Finish[P2] = true.

3. Check P3: Request [1, 0, 0] <= Work [3, 1, 3]. P3 can finish! Work = [3, 1, 3] + [2, 1, 1] = [5, 2, 4]. Finish[P3] = true.

4. Check P1: Request [2, 0, 2] <= Work [5, 2, 4]. P1 can finish! Work = [5, 2, 4] + [2, 0, 0] = [7, 2, 4]. Finish[P1] = true.

5. Check P4: Request [0, 0, 2] <= Work [7, 2, 4]. P4 can finish! Work = [7, 2, 4] + [0, 0, 2] = [7, 2, 6]. Finish[P4] = true.

[FINAL ANSWER: Deadlock Detection Algorithm Result]
Safe Execution Sequence: < P0, P2, P3, P1, P4 >
Conclusion: All Finish[i] == true. The system is NOT in deadlock!

Q2. Consider 5 processes P0-P4 and 3 resources A(10), B(5), C(7). Given snapshot at t0: Allocation and Max matrices. Calculate: (i) Need Matrix, (ii) Safe Sequence, (iii) Can Request by P1 for (1, 0, 2) be granted immediately? [10M]

(Source: OS Units 3,4,5 Problems Bank)

Step 1 – Content of the Need Matrix:

Formula: Need[i][j] = Max[i][j] - Allocation[i][j]

Process	Max (A B C)	Allocation (A B C)	Need = Max - Allocation (A B C)
P0	7 5 3	0 1 0	7 - 0, 5 - 1, 3 - 0 = [7, 4, 3]
P1	3 2 2	2 0 0	3 - 2, 2 - 0, 2 - 0 = [1, 2, 2]
P2	9 0 2	3 0 2	9 - 3, 0 - 0, 2 - 2 = [6, 0, 0]
P3	2 2 2	2 1 1	2 - 2, 2 - 1, 2 - 1 = [0, 1, 1]
P4	4 3 3	0 0 2	4 - 0, 3 - 0, 3 - 2 = [4, 3, 1]

Step 2 – Safety State Verification (Work = [3, 3, 2]):

1. For P1: Need [1, 2, 2] <= Work [3, 3, 2] (True). P1 runs -> Work = [3, 3, 2] + [2, 0, 0] = [5, 3, 2]. Finish[P1] = true.
2. For P3: Need [0, 1, 1] <= Work [5, 3, 2] (True). P3 runs -> Work = [5, 3, 2] + [2, 1, 1] = [7, 4, 3]. Finish[P3] = true.
3. For P4: Need [4, 3, 1] <= Work [7, 4, 3] (True). P4 runs -> Work = [7, 4, 3] + [0, 0, 2] = [7, 4, 5]. Finish[P4] = true.
4. For P0: Need [7, 4, 3] <= Work [7, 4, 5] (True). P0 runs -> Work = [7, 4, 5] + [0, 1, 0] = [7, 5, 5]. Finish[P0] = true.
5. For P2: Need [6, 0, 0] <= Work [7, 5, 5] (True). P2 runs -> Work = [7, 5, 5] + [3, 0, 2] = [10, 5, 7]. Finish[P2] = true.

Step 3 – Resource Request by P1 for [1, 0, 2]:

1. Check: Request [1, 0, 2] <= Need_P1 [1, 2, 2] (True).
2. Check: Request [1, 0, 2] <= Available [3, 3, 2] (True).
3. Pretend Allocation: Available = [3, 3, 2] - [1, 0, 2] = [2, 3, 0]. Allocation_P1 = [2, 0, 0] + [1, 0, 2] = [3, 0, 2]. Need_P1 = [1, 2, 2] - [1, 0, 2] = [0, 2, 0].

Test Safety: P1 Need [0, 2, 0] <= Work [2, 3, 0]. P1 completes -> Work = [2, 3, 0] + [3, 0, 2] = [5, 3, 2]. P3, P4, P0, P2 follow in safe sequence.

[FINAL ANSWER: Banker's Algorithm Solutions]
Need Matrix: P0=[7,4,3], P1=[1,2,2], P2=[6,0,0], P3=[0,1,1], P4=[4,3,1]
System is SAFE with Safe Sequence: < P1, P3, P4, P0, P2 >
Request by P1 for [1, 0, 2] CAN be granted immediately!

UNIT - IV: MEMORY-MANAGEMENT STRATEGIES & MASS STORAGE

SECTION A: 2-MARK QUESTIONS (Short, Direct Answers)

Q1. Define dynamic linking. [2M]

(Source: University Exam 2026)

Dynamic linking is a mechanism where shared library routines (e.g., standard C library) are not combined into the executable image at compile time, but rather loaded into physical RAM and linked dynamically at runtime when the program calls them.

Q2. What is a page fault? [2M]

(Source: University Exam 2026)

A page fault is a hardware interrupt raised by the Memory Management Unit (MMU) when an executing process references a logical page whose valid/invalid bit in the page table is set to "invalid" (indicating the page is on disk and not currently in RAM).

Q3. What is paging in memory management? [2M]
(Source: Model Paper-I)
Paging is a non-contiguous physical memory management scheme that divides logical memory into fixed-size blocks called "Pages" and physical physical memory into identical fixed-size blocks called "Frames". It completely eliminates external fragmentation.

Q4. Define demand paging. [2M]
(Source: Model Paper-I / Mid-2 QB)
Demand paging is a virtual memory implementation where a page is never brought into physical memory until an executing instruction actually references (demands) it. Often called "lazy paging".

Q5. What is thrashing in operating systems? [2M]
(Source: Model Paper-II / Mid-2 QB)
Thrashing is a catastrophic system state where the operating system spends significantly more time swapping pages in and out of disk than executing instructions, causing CPU utilization to drop to near zero.

Q6. Define swapping in memory management. [2M]
(Source: Model Paper-II)
Swapping is the process of temporarily moving an entire idle or blocked process from main memory to backing store (swap space on disk) and later bringing it back into RAM for continued execution.

Q7. Difference between internal fragmentation & external fragmentation. [2M]
(Source: Mid-2 QB)
Internal fragmentation occurs when allocated physical memory block is slightly larger than the requested memory (unused wasted space inside the block, common in paging). External fragmentation occurs when total free memory exists to satisfy a request, but it is divided into non-contiguous small fragments (common in variable-partition contiguous allocation).

Q8. What is Translation Lookaside Buffer (TLB)? [2M]
(Source: Mid-2 QB)
The TLB is a high-speed associative hardware cache inside the MMU that stores recently used page-table translations (Page Number to Frame Number). Searching the TLB takes less than a clock cycle, drastically reducing effective memory access times.

Q9. What is copy-on-write? [2M]
(Source: Mid-2 QB)
Copy-On-Write (COW) is a resource optimization technique used during `fork()`. The parent and child processes initially share the exact same physical pages marked read-only. Only if either process modifies (writes to) a page, a physical copy of that specific page is created.

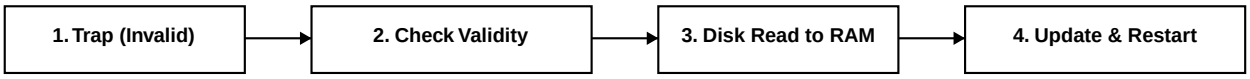
Q10. What is RAID? State its main purpose. [2M]
(Source: Mid-2 QB)
RAID (Redundant Array of Independent Disks) organizes multiple physical hard drives into a single logical storage unit. Its dual purposes are: 1. Reliability (via data redundancy/mirroring/parity) and 2. Performance (via disk striping).

SECTION B: 5-MARK QUESTIONS (Detailed Conceptual Answers)

Q1. Discuss demand paging and describe page fault handling. [5M]
(Source: University Exam 2026 / Model Paper-I / Mid-2 QB)
Demand paging loads pages lazily only upon reference. The page table maintains a Valid/Invalid (v/i) bit. The 6-step page fault handling sequence is:

- **1. Trap:** CPU references page marked invalid; MMU traps to OS kernel.
- **2. Check Validity:** OS checks internal PCB table. If reference was illegal address, abort; if valid but unmapped, page in.
- **3. Find Free Frame:** OS searches the free-frame list in physical RAM.
- **4. Disk Read:** OS schedules disk I/O to read the desired page from backing store into the allocated frame.
- **5. Update Page Table:** When I/O finishes, OS updates page table entry to point to frame and flips bit to "valid" (v).
- **6. Restart Instruction:** CPU restarts the instruction that was interrupted by the page fault.

Fig 4.1: Six Steps in Handling a Page Fault



Q2. Explain contiguous memory allocation and discuss fragmentation issues. [5M]

(Source: University Exam 2026 / Model Paper-I / Mid-2 QB)

In contiguous memory allocation, each process is loaded into a single contiguous section of memory:

- **Relocation & Limit Registers:** The Relocation (Base) register stores smallest physical address; Limit register specifies size. MMU checks: `Logical Address < Limit`; if true, `Physical Address = Logical Address + Base`.
- **Dynamic Storage Allocation Strategies:** 1. **First-Fit:** Allocate first hole large enough (fastest). 2. **Best-Fit:** Allocate smallest hole that is big enough (leaves smallest leftover fragments). 3. **Worst-Fit:** Allocate largest available hole.
- **Fragmentation Problems:** External fragmentation occurs when small dynamic holes cannot satisfy incoming requests. Solved by Compaction (shuffling memory contents) or Paging.

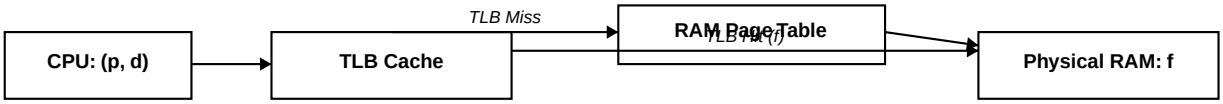
Q3. What is Translation Lookaside Buffer (TLB)? Explain its working with a diagram. [5M]

(Source: University Exam 2026)

Standard paging requires two memory accesses per data reference (one for page table, one for data), cutting memory speed in half. The TLB solves this:

- **Working Mechanism:** The CPU generates logical address (p, d). MMU sends page number p simultaneously to all TLB entries in parallel hardware. If p is found (TLB Hit), frame number f is retrieved in < 1 ns. If p is not found (TLB Miss), MMU accesses page table in RAM, loads (p, f) into TLB, and accesses data.

Fig 4.2: Paging Hardware Architecture with TLB



Q4. Define thrashing. Why does it occur, and how can it be controlled? [5M]

(Source: Model Paper-II / Mid-2 QB)

Thrashing happens when processes do not have enough frames to hold their active working sets:

- **Cause:** When CPU utilization drops, the OS long-term scheduler introduces more processes (increasing degree of multiprogramming). These processes need more frames, taking frames from existing processes, triggering continuous page faults. CPU sits idle waiting for disk paging.
- **Working-Set Model Control:** Peter Denning's model defines Δ (working set window). A process's working set WSS_i is the set of pages referenced in the last Δ time units. Total demand $D = \sum WSS_i$. If $D >$ Total available RAM, the OS suspends one entire process, freeing its frames to prevent thrashing.

Q5. Differentiate between swapping and paging with suitable examples. [5M]

(Source: Model Paper-I)

Comparison Basis	Swapping	Paging
Unit of Transfer	Entire process image moved between RAM and disk	Individual fixed-size blocks (pages, typically 4 KB)
Fragmentation	Suffers from external fragmentation	No external fragmentation; small internal fragmentation
Memory Requirement	Process must fit entirely into contiguous RAM	Process can be scattered across non-contiguous frames
I/O Overhead	High I/O overhead moving megabytes of data	Low I/O overhead moving single small page on demand

SECTION C: 10-MARK / LAQ QUESTIONS (Comprehensive University Answers)

Q1. Explain Paging hardware with TLB and describe how Effective Memory Access Time (EMAT) is calculated with neat diagrams. [10M]

(Source: Mid-2 QB)

Paging hardware translates logical addresses generated by CPU into physical addresses without requiring contiguous allocation.

1. Mathematical Address Translation Scheme
Logical address generated by CPU is divided into two parts: Page Number (p) and Page Offset (d). If logical address space is 2^m and page size is 2^n bytes, high-order $m-n$ bits designate page number p, and low-order n bits designate offset d.

Physical address = $(f \times \text{Page Size}) + d$, where f is Frame Number retrieved from page table.

2. Effective Memory Access Time (EMAT) Formula
Let α be the TLB Hit Ratio (percentage of times page is found in TLB).

Let ϵ be the TLB Search Time.

Let M be the Main Memory Access Time.

- **If TLB Hit occurs:** Access Time = $\epsilon + M$ (search TLB + fetch data from physical frame).
- **If TLB Miss occurs:** Access Time = $\epsilon + M + M$ (search TLB + fetch frame from page table + fetch data).
- **Effective Memory Access Time:** $\text{EMAT} = \alpha \times (\epsilon + M) + (1 - \alpha) \times (\epsilon + 2M) = \epsilon + (2 - \alpha) \times M$.

SECTION D: NUMERICAL PROBLEMS (Step-by-Step Solved Calculations)

Q1. Consider a paging hardware with a TLB. Entire page table is in memory. TLB search time = 10 ms, physical memory access time = 80 ms, TLB hit ratio = 0.6. Calculate Effective Memory Access Time (EMAT). [5M]
(Source: OS Units 3,4,5 Problems Bank)

Step 1 – Given Data:
TLB Search Time (ϵ) = 10 ms; Memory Access Time (M) = 80 ms; Hit Ratio (α) = 0.6; Miss Ratio ($1 - \alpha$) = 0.4.

Step 2 – Formula:
$\text{EMAT} = \alpha \times (\epsilon + M) + (1 - \alpha) \times (\epsilon + M + M)$

Step 3 – Step-by-Step Calculation:
Case 1 (Hit Time): $\epsilon + M = 10 + 80 = 90\text{ ms}$

Case 2 (Miss Time): $\epsilon + 2M = 10 + 80 + 80 = 170\text{ ms}$

$\text{EMAT} = (0.6 \times 90) + (0.4 \times 170) = 54 + 68 = 122\text{ ms}$

[FINAL ANSWER: Effective Memory Access Time (EMAT)]
Effective Memory Access Time = 122 milliseconds

Q2. Given memory partitions of 500 KB, 100 KB, 200 KB, 300 KB, and 600 KB (in order). How would First-Fit, Best-Fit, and Worst-Fit place processes of 212 KB, 417 KB, 112 KB and 426 KB (in order)? Which is most efficient? [5M]
(Source: OS Units 3,4,5 Problems Bank)

1. First-Fit Allocation:
• P1 (212 KB) -> Placed in partition 500 KB (leftover: 288 KB).

• P2 (417 KB) -> Placed in partition 600 KB (leftover: 183 KB).

• P3 (112 KB) -> Placed in partition 288 KB (leftover of 500 KB) or next fit 200 KB (leftover: 88 KB).

• P4 (426 KB) -> Must wait! No hole ≥ 426 KB available.

2. Best-Fit Allocation:
• P1 (212 KB) -> Placed in 300 KB (closest fit, leftover: 88 KB).

• P2 (417 KB) -> Placed in 500 KB (closest fit, leftover: 83 KB).

• P3 (112 KB) -> Placed in 200 KB (closest fit, leftover: 88 KB).

• P4 (426 KB) -> Placed in 600 KB (closest fit, leftover: 174 KB). All processes allocated!

3. Worst-Fit Allocation:

• P1 (212 KB) -> Placed in largest: 600 KB (leftover: 388 KB).

• P2 (417 KB) -> Placed in 500 KB (leftover: 83 KB).

• P3 (112 KB) -> Placed in 388 KB (leftover: 276 KB).

• P4 (426 KB) -> Must wait! No remaining partition ≥ 426 KB.

[FINAL ANSWER: Memory Partition Placement Strategy Result]
Best-Fit is the MOST EFFICIENT algorithm: It successfully allocates all four processes!
First-Fit and Worst-Fit fail to allocate process P4 (426 KB).

Q3. Consider page reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 with 3 frames. Trace FIFO, Optimal, and LRU algorithms. [10M]

(Source: OS Units 3,4,5 Problems Bank)

Trace of page faults with 3 frames:

• **FIFO Algorithm:** Replaces oldest loaded page. Total Page Faults = 15; Page Fault Rate = 15/20 = 75%.

• **Optimal Algorithm:** Replaces page that will not be used for longest future period. Total Page Faults = 9; Page Fault Rate = 9/20 = 45%.

• **LRU Algorithm:** Replaces page not used for longest past period. Total Page Faults = 12; Page Fault Rate = 12/20 = 60%.

Algorithm	Total Page References	Frames	Page Faults	Page Hits	Fault Percentage
FIFO	20	3	15	5	75.0%
Optimal (OPT)	20	3	9	11	45.0%
Least Recently Used (LRU)	20	3	12	8	60.0%

[FINAL ANSWER: Page Replacement Algorithms Comparison]
Optimal produces fewest page faults (9 faults).
LRU produces 12 faults (good practical approximation).
FIFO produces most page faults (15 faults).

Q4. Disk with 200 cylinders (0-199), initial head at 53. Queue: 98, 183, 37, 122, 14, 124, 65, 67. Calculate total arm movement for: (i) FCFS, (ii) SSTF, (iii) SCAN (towards 199), (iv) LOOK, (v) C-SCAN, (vi) C-LOOK. [10M]

(Source: Mid-2 QB)

1. FCFS (First Come First Served):

Head sequence: 53 -> 98 -> 183 -> 37 -> 122 -> 14 -> 124 -> 65 -> 67.

Total head movement = |98-53| + |183-98| + |37-183| + |122-37| + |14-122| + |124-14| + |65-124| + |67-65| = 45 + 85 + 146 + 85 + 108 + 110 + 59 + 2 = 640 cylinders.

2. SSTF (Shortest Seek Time First):

From 53, nearest is 65, then 67, then 37, then 14, then 98, then 122, then 124, then 183.

Total head movement = (67 - 53) + (67 - 14) + (183 - 14) = 14 + 53 + 169 = 236 cylinders.

3. SCAN (Moves towards 199, then reverses):

Sequence: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 199 (end) -> 37 -> 14.

Total movement = (199 - 53) + (199 - 14) = 146 + 185 = 331 cylinders.

4. LOOK (Moves towards 183, then reverses to 14, does not go to 199):

Total movement = (183 - 53) + (183 - 14) = 130 + 169 = 299 cylinders.

5. C-SCAN (Circular SCAN to 199, jumps to 0, serves up to 37):

Total movement = (199 - 53) + (199 - 0) + (37 - 0) = 146 + 199 + 37 = 382 cylinders.

6. C-LOOK (Circular LOOK to 183, jumps to 14, serves up to 37):

Total movement = (183 - 53) + (183 - 14) + (37 - 14) = 130 + 169 + 23 = 322 cylinders.

[FINAL ANSWER: Comprehensive Disk Scheduling Results Summary]
FCFS = 640 cylinders
SSTF = 236 cylinders (Lowest movement)
SCAN = 331 cylinders
LOOK = 299 cylinders
C-SCAN = 382 cylinders
C-LOOK = 322 cylinders

UNIT - V: FILE SYSTEM INTERFACE, IMPLEMENTATION & PROTECTION

SECTION A: 2-MARK QUESTIONS (Short, Direct Answers)

Q1. List the attributes of a file. [2M]
(Source: University Exam 2026)
A file is an abstract data type. Standard file attributes maintained by the OS directory entry include: 1. Name, 2. Identifier (unique numeric ID), 3. Type, 4. Location (pointer to device block), 5. Size, 6. Protection (access permissions), and 7. Time, date, and user identification.

Q2. Define free space management. [2M]
(Source: University Exam 2026)
Free space management is the OS subsystem that maintains a free-space list tracking all unallocated physical disk blocks so that incoming file write and expansion requests can be allocated contiguous or linked storage quickly.

Q3. What is a directory structure in file systems? [2M]
(Source: Model Paper-I)
A directory structure is a symbol table that translates human-readable file names into their corresponding file control blocks (FCBs) or inodes containing disk block addresses.

Q4. State the goal of protection in an operating system. [2M]
(Source: Model Paper-I / Mid-2 QB)
The primary goal of protection is to provide a mechanism that strictly controls the access of processes or users to resources defined by a computer system, preventing unauthorized read, write, or modification operations.

Q5. What is an access matrix in protection? [2M]
(Source: Model Paper-II / Mid-2 QB)
An access matrix is an abstract security model where rows represent Domains (users/processes) and columns represent Objects (files/printers). Entry `Access(D_i, O_j)` defines the precise set of access rights process executing in domain D_i has on object O_j (e.g., `{read, write}`).

Q6. Define file mounting. [2M]
(Source: Model Paper-II)
File mounting is the operation by which an external storage volume or partition filesystem is attached to an existing directory location (called the mount point) in the system's active directory tree, making its files accessible.

Q7. List different file access methods. [2M]
(Source: Mid-2 QB)
The three primary file access methods are: 1. Sequential Access (bytes read in chronological order); 2. Direct / Relative Access (random access by record number); 3. Indexed Access (index lookup to jump directly to target record).

SECTION B: 5-MARK QUESTIONS (Detailed Conceptual Answers)

Q1. Discuss different access methods of a file system with examples. [5M]
(Source: University Exam 2026 / Model Paper-I / Mid-2 QB)
Operating systems provide three fundamental mechanisms for accessing file data:

Access Method	Working Mechanism	Typical Use Case
Sequential Access	Data read/written strictly in order from start to end; pointer moves forward automatically (`read_next`, `write_next`)	Compilers, audio/video streaming, text editors processing log files
Direct (Relative) Access	File viewed as numbered logical records (`read n`, `write n`); pointer jumps directly to block without reading predecessors	Relational database systems (RDBMS) querying specific row ID
Indexed Access	An index file containing search key and block pointers is searched first; then direct pointer retrieves file record	Large enterprise data warehouses and file search engines

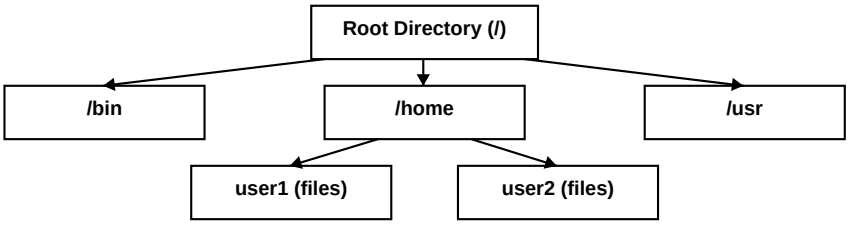
Q2. Explain the various types of directory structures. [5M]

(Source: Model Paper-I)

A directory structure organizes file references within a file system:

- **1. Single-Level Directory:** All files contained in one root directory. Simple, but suffers from naming collisions (no two files can share same name) and poor organization.
- **2. Two-Level Directory:** Each user receives an independent Master File Directory (MFD) and User File Directory (UFD). Prevents naming collisions between users, but does not allow file grouping.
- **3. Tree-Structured Directory:** Directory can contain files and subdirectories, forming an arbitrary-depth tree rooted at `/` (or `C:\`). Standard model in Linux and Windows.
- **4. Acyclic-Graph Directory:** Allows directories to share subdirectories and files via links (hardlinks/symlinks) without forming cycles.

Fig 5.1: Tree-Structured Directory Model



Q3. What is free space management? Explain any two methods with examples. [5M]

(Source: Model Paper-II)

To track unallocated disk blocks, operating systems implement one of four free-space tracking methods:

- **1. Bit Vector (Bit Map):** Free space represented by a bit string where each bit represents a physical disk block: bit is `0` if block allocated, `1` if block free. Example: `1100111...` means blocks 0, 1, 4, 5 are free. Advantage: Extremely fast and compact; hardware bit-finding instructions find first free block in one cycle.
- **2. Linked Free List:** All free disk blocks are linked together via pointers. Free block 0 points to free block 1, which points to free block 4. Advantage: No memory waste; but traversing the list requires multiple slow disk I/O reads.
- **3. Grouping & Counting:** Grouping stores addresses of N free blocks in first free block; Counting records starting address and contiguous block run length.

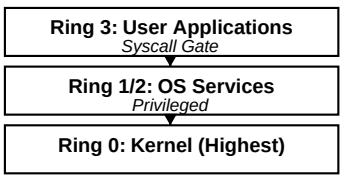
Q4. Explain protection rings and their role in operating-system protection. [5M]

(Source: Mid-2 QB)

Protection rings are hierarchical security hardware domains (e.g., x86 rings 0 through 3) arranged as concentric circles:

- **Ring 0 (Kernel / Most Privileged):** Contains OS kernel, device drivers, and core memory management. Full access to hardware.
- **Ring 1 & 2 (OS Services / Device Drivers):** Optional rings used by specialized virtualization hypervisors or OS subsystems.
- **Ring 3 (User Applications / Least Privileged):** Standard user applications execute here. Any direct access to hardware or Ring 0 memory causes a general protection fault.

Fig 5.2: Hierarchical Protection Rings Architecture



SECTION C: 10-MARK / LAQ QUESTIONS (Comprehensive University Answers)

Q1. Explain file allocation methods in detail: Contiguous, Linked, and Indexed allocation with neat sketches, advantages, and disadvantages. [10M]

(Source: University Exam 2026 / Model Paper-I / Mid-2 QB)

The allocation method defines how physical disk blocks are assigned to files:

1. Contiguous Allocation

Each file occupies a set of contiguous blocks on disk. Directory entry stores only starting block number and file length in blocks.

- **Advantages:** Fastest performance for sequential and direct access; only single seek required to begin reading.
- **Disadvantages:** Causes severe external fragmentation as files are created and deleted; file cannot grow unless adjacent blocks are free.

2. Linked Allocation

Each file is a linked list of disk blocks scattered anywhere on disk. Each block contains data plus a disk pointer to the next block. Directory entry points to the first and last blocks.

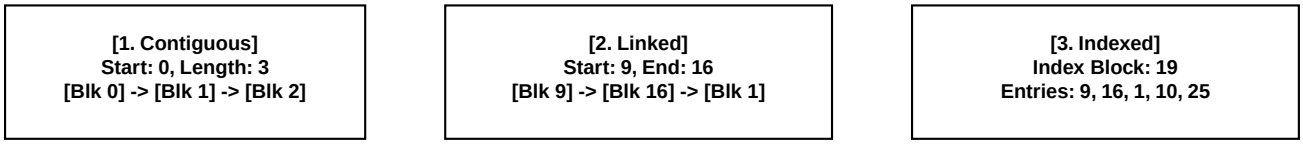
- **Advantages:** Zero external fragmentation; file can grow dynamically indefinitely.
- **Disadvantages:** Only supports sequential access (cannot jump to middle without traversing all pointers); pointer overhead eats space (e.g., 4 bytes per 512-byte block); single lost pointer corrupts rest of file. (Solved by FAT: File Allocation Table).

3. Indexed Allocation

Brings all pointers together into a single dedicated Index Block. The i-th entry in index block points to the i-th block of the file.

- **Advantages:** Supports direct random access efficiently without external fragmentation.
- **Disadvantages:** Index block overhead for very small files; for very large files, single index block is too small (solved by Multi-Level Indexing or UNIX Inodes with direct, single-indirect, double-indirect, and triple-indirect blocks).

Diagram 1 (Standard Exam Diagram): Comparison of Three File Allocation Schemes



Q2. Explain Protection and Security mechanisms in Operating Systems (Domain, Access Matrix, Access Lists, Capability Lists). [10M]

(Source: Model Paper-I / Model Paper-II / Mid-2 QB)

Protection ensures that each active process can access only the resources for which it has explicit authorization.

1. Principle of Least Privilege

Programs, users, and systems should be given just enough privileges to perform their assigned tasks, minimizing the potential damage from security compromise.

2. Access Matrix Model

The access matrix is a 2D grid: Rows = Protection Domains (D_1, D_2), Columns = Objects ($F_1, F_2, \text{Printer}$).

Domain	File F1	File F2	File F3	Printer
Domain D1	Read	Read, Write	—	—
Domain D2	Read	—	—	Print
Domain D3	—	—	Read, Execute	—

3. Practical Implementation of Access Matrix

A full access matrix is sparse (mostly empty cells). OS implements it via two decomposed structures:

- **Access Control Lists (ACLs - Column-wise):** Each object stores a list of domains and allowed rights. E.g., `File F1: { (D1, Read), (D2, Read) }`. Used in UNIX (`chmod 755`).

• **Capability Lists (Row-wise):** Each domain carries an unforgeable ticket (capability) listing accessible objects. E.g., `Domain D1: { (F1, Read), (F2, Read/Write) }`. Fast validation, but revocation is hard.

[FINAL ANSWER: Access Matrix Implementation Summary]
ACLs (Access Control Lists) = Column-wise decomposition (Attached to Objects)
Capability Lists = Row-wise decomposition (Attached to Domains / Processes)

APPENDIX: FINAL EXAM REVISION CHECKLISTS & FORMULAS

1. MASTER LIST OF ESSENTIAL EXAM DIAGRAMS TO PRACTICE

To score maximum marks in the university examination, practice drawing these 12 core black-and-white technical diagrams before the exam:

Unit	Diagram Title	Key Components to Draw Accurately
Unit I	Dual-Mode Operation Switching	User Mode (Mode 1), Kernel Mode (Mode 0), Trap / Syscall, Return to User
Unit I	Layered vs Microkernel OS Structure	Layer 0 to N hierarchy; Microkernel user-level servers communicating via IPC
Unit I	5-Stage System Boot Sequence	ROM BIOS (POST) -> MBR Boot Block -> Bootloader (GRUB) -> Kernel RAM -> Init
Unit II	5-State Process State Diagram	New, Ready, Running, Waiting, Terminated with admitted, dispatch, timeout, I/O arrows
Unit II	Process Control Block (PCB) Structure	PID, State, Program Counter, CPU Registers, Memory Limits, Open File List
Unit II	Multithreading Models Mapping	Many-to-One, One-to-One, Many-to-Many mapping user threads to kernel threads
Unit III	Resource Allocation Graph (RAG)	Processes (circles), Resources (rectangles with dots), Request & Assignment edges
Unit III	Dining Philosophers Problem Setup	Circular table, 5 philosophers, 5 chopsticks, central rice bowl
Unit IV	Paging Hardware with TLB	CPU (p, d), TLB cache hit/miss, Page Table in RAM, Frame Number (f), Physical Memory
Unit IV	6-Step Page Fault Handling Routine	Trap, PCB check, Free frame lookup, Disk page fetch, Page table update, Restart
Unit V	File Allocation Methods	Contiguous (blocks array), Linked (block pointers), Indexed (index block table)
Unit V	Hierarchical Protection Rings	Concentric rings: Ring 0 (Kernel), Ring 1/2 (Services), Ring 3 (User Applications)

2. NUMERICAL PROBLEM FORMULA CHEAT SHEET

Quick formula reference for all numerical problems in the examination:

A. CPU Scheduling Formulas:

• **Turnaround Time (TAT):** $\text{TAT} = \text{Completion Time (CT)} - \text{Arrival Time (AT)}$

• **Waiting Time (WT):** $\text{WT} = \text{Turnaround Time (TAT)} - \text{Burst Time (BT)}$

• **Average TAT / WT:** $\text{Average} = \frac{\sum \text{Values}}{\text{Number of Processes}}$

B. Banker's Algorithm & Deadlock Avoidance:

• **Need Matrix Formula:** $\text{Need}[i][j] = \text{Max}[i][j] - \text{Allocation}[i][j]$

• **Safety Test Condition:** Find process where $\text{Finish}[i] == \text{false}$ and $\text{Need}[i] \le \text{Work}$. Then $\text{Work} = \text{Work} + \text{Allocation}[i]$.

C. Paging & Memory Access Time with TLB:

• **Effective Memory Access Time:** $\text{EMAT} = \alpha \times (\epsilon + M) + (1 - \alpha) \times (\epsilon + 2M)$ where α = hit ratio, ϵ = TLB search time, M = memory access time.

D. Disk Scheduling Arm Movement:

• **Total Head Movement:** $\text{Total Distance} = \sum |\text{Current Track} - \text{Next Track}|$

• **SCAN Rule:** Moves in current direction until physical end of disk (track 0 or track 199), then reverses.

• **LOOK Rule:** Moves in current direction only until the furthest pending request, then reverses.

[FINAL ANSWER: Golden Rules for Operating Systems Exam Success]

1. **Always draw a neat, labeled block diagram for every single question (even 2M).**
2. **For CPU scheduling and disk scheduling, always draw the complete Gantt chart / timeline.**
3. **For page replacement, always present the complete reference string table with hits/faults marked.**
4. **In comparison questions, always use a structured table with clear basis of comparison.**